

Использование GitHub для управления временем разработчика

1. Выбор сервиса

Для анализа был выбран GitHub. Он позволяет хранить код и документы, создавать задачи, назначать сроки и отслеживать выполнение работы. На примере заданий практики рассмотрены Issues, Projects, Milestones и связь задач с изменениями в репозитории.

2. Создание и планирование задач

В GitHub отдельная задача оформляется как Issue. В ней указываются название, описание результата и исполнитель. Метки помогают разделять задачи, например на исследование, программирование и оформление отчёта.

Для задания 2.2 подготовлен пример карточки:

Название: «Подготовить подборку материалов по Julia».

Результат: аннотированные источники и три примера кода с пояснениями.

План: найти материалы, написать аннотации, подготовить примеры, проверить отчёт.

Контрольный срок: 25 сентября. Предварительная оценка: 3 часа.

В комплект также включена форма `practice-task.yml`. После загрузки в папку `.github/ISSUE_TEMPLATE` она позволит создавать задачи с одинаковыми полями: номер задания, результат, оценка времени и критерии готовности.

3. Контроль сроков и выполнения

Projects позволяет представить задачи в виде таблицы, доски или дорожной карты. Для практики подходит доска с тремя колонками: «Нужно сделать», «В работе» и «Готово». В проект можно добавить поля для срока, приоритета и оценки времени.

Пример распределения оставшихся работ:

- 24 сентября - анализ возможностей GitHub;
- 25 сентября - подборка материалов по Julia;
- 26 сентября - демонстрация команд Git;
- 27 сентября - проверка отчётов и ссылок;
- 28 сентября - публикация и сдача практики.

Это плановые сроки. Для общего срока сдачи можно создать Milestone «Сдача практики» с датой 28 сентября и включить в него связанные задачи. Веха показывает количество открытых и завершённых задач.

Изменения в файлах можно связать с задачей через pull request. Например, запись `Closes #12` в описании запроса закрывает связанную задачу после его объединения с основной веткой. Номер 12 здесь приведён как пример.

В результате анализа подготовлены образец карточки, форма Issue и план работ. Они сохранены локально; доска Projects и задачи на сайте отдельно не создавались.

4. Достоинства и недостатки

Главное достоинство GitHub - возможность хранить задачи рядом с кодом и отчётами. Для каждой работы можно указать исполнителя, обсудить результат и увидеть историю изменений. Доска помогает определить, чем нужно заниматься сейчас, а веха - контролировать общий срок.

Недостаток заключается в необходимости первоначальной настройки. Нужно разобраться в задачах, проектах и вехах, а затем регулярно обновлять статусы. Фактически затраченное время в рассмотренной схеме приходится учитывать вручную.

5. Вывод

GitHub удобен для планирования задач разработчика, особенно если материалы уже хранятся в репозитории. Для небольшой практики достаточно списка задач, трёх статусов и сроков. Такая организация помогает видеть оставшуюся работу и связывать завершение задачи с конкретным результатом.

Источники: официальная документация GitHub по Projects, Milestones и связи pull request с Issue. Дата обращения: 24.09.2026.

GitHub Docs - Projects

GitHub Docs - Milestones

GitHub Docs - связь pull request и Issue